

Python Topics to Master Azure Data Engineer Roadmap

From Core Python to PySpark, Azure SDKs, Delta Lake & Beyond

#	Category	Priority
1	Core Python Fundamentals	Non-Negotiable
2	Data Engineering Libraries	Critical
3	Azure-Specific Python SDKs	Critical
4	Data Pipeline & Orchestration	High
5	Data Quality & Testing	High
6	Performance & Best Practices	High
7	Supporting Skills	Important
8	Suggested Learning Order	Roadmap

Section 1 — Core Python Fundamentals (Non-Negotiable)

1.1 Data Structures & Pythonic Code

- Lists, dicts, sets, tuples — and when to use each for performance
- List / dict / set comprehensions — concise, readable transformations
- Generators and iterators — critical for memory-efficient large data processing
- Unpacking, *args, **kwargs — flexible function signatures
- Context managers (with statement) — safe resource handling for files & connections
- Slicing, sorting, enumerate, zip, any, all — everyday Pythonic patterns

1.2 Object-Oriented & Functional Programming

- Classes, inheritance, __dunder__ / magic methods (__init__, __repr__, __len__, etc.)
- Properties, class methods, static methods
- map(), filter(), reduce() — functional transformations
- Decorators and closures — used heavily in Airflow and custom pipeline frameworks
- functools (lru_cache, partial, wraps) and itertools (chain, groupby, islice)
- Abstract base classes (ABC) — for building reusable pipeline interfaces

1.3 Error Handling & Logging

- try / except / finally / else — structured exception handling
- Custom exception classes — domain-specific error hierarchies for pipelines
- logging module (NOT print()) — structured logging for observability in Azure Monitor
- Log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Retry patterns using tenacity or custom exponential backoff decorators

1.4 File & I/O Operations

- Reading and writing CSV, JSON, Parquet, Avro, XML
- os, pathlib, shutil — file system operations and path management
- Environment variables and config management (python-dotenv, configparser, pydantic settings)
- Binary vs text mode — understanding encoding (UTF-8, Latin-1)
- Streaming large files line by line instead of loading fully into memory

■ Master pathlib over os.path — it is the modern, cleaner standard for all path operations.

Section 2 — Data Engineering Libraries (Critical)

2.1 Pandas — Deep Mastery Required

- DataFrames and Series — creation, inspection, shape, dtypes
- Indexing: loc (label-based), iloc (position-based), boolean filtering
- GroupBy: split-apply-combine pattern, agg, transform, apply
- merge / join / concat — left, right, inner, outer joins
- pivot_table, melt, stack, unstack — reshaping data
- Handling nulls: isna(), fillna(), dropna(), interpolate()
- Type casting, string operations (.str accessor), datetime operations (.dt accessor)
- Deduplication: drop_duplicates(), duplicated()
- Memory optimisation: chunking with chunksize, downcasting dtypes (int64 -> int32)
- Reading and writing: Parquet, CSV, JSON, Excel, SQL
- Method chaining for readable transformation pipelines

■ *Always profile memory usage with `df.memory_usage(deep=True)` before loading large datasets.*

2.2 PySpark — Most Important for Azure DE

- RDD vs DataFrame vs Dataset — know the differences and when each applies
- Lazy evaluation: transformations vs actions — understanding the execution model
- Core transformations: select, filter, withColumn, groupBy, agg, join, union
- Window functions: row_number, rank, lead, lag, sum over partitions
- Schema definition and enforcement using StructType / StructField
- Reading and writing Delta, Parquet, CSV from ADLS Gen2
- Broadcasting small tables to avoid shuffle joins
- Caching and persistence: cache(), persist(), unpersist()
- repartition() vs coalesce() — when to increase vs reduce partitions
- UDFs — how to write them AND why to avoid them (serialisation overhead)
- Pandas UDFs (vectorised) — the performant alternative to standard UDFs
- Structured Streaming: readStream, writeStream, trigger, watermarking
- Spark SQL: spark.sql(), createOrReplaceTempView()
- Handling skewed data: salting technique, AQE (Adaptive Query Execution)
- Databricks-specific: spark.conf.set(), display(), dbutils integration

■ *The single highest-ROI PySpark skill: understanding shuffle operations and how to minimise them.*

2.3 Polars — Increasingly Relevant

- Lazy vs eager evaluation — similar concept to Spark but single-node
- Basic transformations: select, filter, with_columns, group_by, join
- Reading and writing Parquet, CSV, JSON
- When to use Polars over Pandas: large single-node files, speed-critical transforms

Section 3 — Azure-Specific Python SDKs (Critical)

3.1 Key Azure SDK Packages

SDK / Package	Purpose
azure-storage-blob	Read, write, list, delete blobs in Blob Storage & ADLS Gen2
azure-storage-file-datalake	ADLS Gen2 file system operations (directories, ACLs)
azure-identity	Authentication — DefaultAzureCredential, ManagedIdentityCredential
azure-keyvault-secrets	Fetch secrets from Key Vault at runtime — never hardcode creds
azure-servicebus	Messaging: send/receive messages, dead-letter queues
azure-eventhub	Produce and consume events from Azure Event Hub streams
azure-mgmt-datafactory	Trigger, monitor, and manage ADF pipelines programmatically
azure-mgmt-synapse	Manage Synapse workspaces and pipelines via Python
azure-synapse-spark	Submit Spark jobs to Synapse Spark pools
pyodbc / pymssql	Connect to Azure SQL / Synapse Dedicated Pool from Python
azure-monitor-ingestion	Send custom logs and metrics to Azure Monitor / Log Analytics

3.2 Critical Pattern — Passwordless Authentication

- Always use DefaultAzureCredential — it auto-selects the right credential (Managed Identity in Azure, CLI locally)
- Never hardcode connection strings or keys — always pull from Key Vault at runtime
- Use Managed Identity for all Azure-to-Azure service communication
- Understand the credential chain: Env -> Workload Identity -> Managed Identity -> Azure CLI -> ...
- SecretClient from azure-keyvault-secrets for fetching secrets in pipelines

■ *DefaultAzureCredential works identically locally (via az login) and in production (via Managed Identity) — write once, run anywhere.*

Section 4 — Data Pipeline & Orchestration (High Priority)

4.1 Databricks-Specific Python

- `dbutils.fs` — list, copy, move, delete files on DBFS and ADLS Gen2
- `dbutils.secrets` — retrieve Key Vault secrets inside notebooks
- `dbutils.widgets` — parameterise notebooks for ADF / pipeline calls
- `dbutils.notebook.run()` — orchestrate multi-notebook workflows
- `%run` vs `dbutils.notebook.run()` — know the scope difference
- Delta Lake API: `DeltaTable.forPath()`, `merge()`, `update()`, `delete()`, `history()`
- Unity Catalog: `spark.catalog`, GRANT/REVOKE via Python/SQL
- Cluster configuration via Python: `SparkConf`, `spark.conf.set()`

4.2 Azure Data Factory + Python

- Triggering ADF pipelines: `create_run()` via `azure-mgmt-datafactory`
- Monitoring pipeline run status: `get_pipeline_run()`, `activity_runs_query_by_pipeline_run()`
- Passing parameters to ADF pipelines from Python
- Using ADF Python activity to run custom scripts in pipelines
- Webhook patterns — calling Python APIs from ADF Web Activity

4.3 Apache Airflow (used with Azure)

- DAG structure: tasks, dependencies, `schedule_interval`, `start_date`
- Core operators: `PythonOperator`, `BashOperator`, `BranchPythonOperator`
- Azure operators: `AzureDataFactoryRunPipelineOperator`, `WasbHook`, `MsSqlHook`
- XComs — passing data between tasks
- TaskFlow API (`@task` decorator) — modern, cleaner DAG authoring
- Connection management — storing Azure credentials as Airflow connections
- SLA misses, retries, `on_failure_callback` — production-grade DAG design
- Dynamic DAG generation from config files or database tables

■ *In Azure environments, Airflow is often hosted via Azure Managed Airflow (within ADF) or self-hosted on AKS.*

Section 5 — Data Quality & Testing (High Priority)

5.1 Great Expectations

- Defining Expectation Suites — rules on columns, nullability, ranges, uniqueness
- Running validation checkpoints in pipelines
- Data Docs — auto-generated HTML quality reports
- Integrating GE checkpoints into Databricks notebooks and ADF pipelines
- Custom expectations for domain-specific business rules

5.2 pytest — Unit Testing Pipeline Logic

- Writing test functions, fixtures, parametrize decorator
- Testing Pandas transformation functions in isolation
- Testing PySpark transformations locally using pytest + pyspark
- Mocking Azure SDK calls with unittest.mock.patch and MagicMock
- Testing for edge cases: empty DataFrames, null values, type mismatches
- Test coverage with pytest-cov
- Structuring a testable codebase: separate transformation logic from I/O

5.3 Data Validation Patterns

- Schema validation using pydantic or pandera
- Row count reconciliation between source and target
- Checksum / hash-based validation for critical financial data
- Null rate monitoring — alerting when null % exceeds threshold
- Referential integrity checks between dimension and fact tables

■ *Write tests BEFORE writing pipeline code — it forces you to design clean, testable transformation functions.*

Section 6 — Performance & Best Practices (High Priority)

6.1 Memory & Efficiency

- Chunking large files: `pd.read_csv(chunksize=)` — never load 10GB into memory at once
- Vectorised operations over row-level loops — 100x faster in Pandas/NumPy
- Generators for lazy data pipelines — yield instead of return
- Efficient data types: use `int32` over `int64`, `float32` over `float64` where safe
- Using Parquet over CSV — columnar storage, predicate pushdown, compression
- Avoiding object dtype in Pandas — use Categorical for low-cardinality strings

6.2 Profiling & Debugging

- cProfile and pstats — identifying slow functions in pipelines
- `memory_profiler` — line-by-line memory usage tracking
- `timeit` — micro-benchmarking critical code paths
- Spark UI — understanding stages, tasks, shuffle read/write, skew
- `py-spy` — sampling profiler for production Python processes

6.3 Concurrency & Parallelism

- Multiprocessing vs multithreading — understand the GIL and when it matters
- `concurrent.futures` (`ThreadPoolExecutor`, `ProcessPoolExecutor`) — parallel API calls
- `asyncio` and `aiohttp` — `async/await` for concurrent REST API ingestion
- When NOT to use parallelism — I/O-bound vs CPU-bound tasks
- Spark's inherent parallelism vs Python-level parallelism — avoid double parallelism

■ For Azure REST API ingestion (e.g., paginated API to ADLS), `asyncio` + `aiohttp` can give 10-50x throughput improvement over sequential requests.

Section 7 — Supporting Skills (Important)

7.1 SQL from Python

- pyodbc — connecting to Azure SQL, Synapse Dedicated Pool, SQL Server
- sqlalchemy — ORM and engine-based connections, connection pooling
- pandas.read_sql() and to_sql() — moving data between DataFrames and SQL
- Parameterised queries — always use ? or :param to prevent SQL injection
- Connection string management via Key Vault, never hardcoded

7.2 REST API Consumption

- requests library — GET, POST, headers, authentication, error handling
- httpx — modern async-capable HTTP client
- Pagination patterns: offset-based, cursor-based, link-header-based
- Rate limiting — implementing backoff and retry logic
- OAuth2 / Bearer token flows for API authentication
- Parsing and flattening nested JSON responses into flat DataFrames

7.3 Schema & Format Management

- pyarrow — reading/writing Parquet, schema inspection and casting
- fastavro — reading and writing Avro files with schema registry
- json and orjson — JSON parsing and serialisation
- Schema evolution strategies — backward/forward compatibility

7.4 Dev Environment & Packaging

- Virtual environments: venv, conda — isolate project dependencies
- poetry — modern dependency management and packaging
- requirements.txt and pyproject.toml — dependency pinning for reproducibility
- Git fundamentals — branching, commits, PRs, .gitignore for notebooks and secrets
- Pre-commit hooks — black, flake8, isort for code quality enforcement
- Docker basics — containerising Python pipeline code for AKS / Azure Container Instances

Section 8 — Suggested Learning Order & Roadmap

Phase	Topics	Timeline
Phase 1 Foundation	Core Python: data structures, OOP, error handling, file I/O, comprehensions, generators	Weeks 1-3
Phase 2 Data Libraries	Pandas deep dive: groupby, merge, reshape, memory optimisation. Polars basics.	Weeks 4-6
Phase 3 PySpark	PySpark DataFrames, transformations, joins, window functions, Streaming basics, Spark SQL	Weeks 7-10
Phase 4 Azure SDKs	azure-storage-blob, azure-identity, azure-keyvault-secrets, DefaultAzureCredential pattern	Weeks 11-12
Phase 5 Delta Lake	Databricks dbutils, Delta Lake API, MERGE, time travel, Z-Order, OPTIMIZE, VACUUM, CDF	Weeks 13-15
Phase 6 Testing & Quality	pytest, mocking Azure SDKs, Great Expectations, pandera schema validation	Weeks 16-17
Phase 7 Orchestration	Apache Airflow DAGs, TaskFlow API, Azure operators, ADF Python SDK integration	Weeks 18-20
Phase 8 Advanced	asyncio, multiprocessing, profiling, Docker, Unity Catalog, Data Vault patterns	Weeks 21+

■ *The single highest-ROI investment: PySpark + Delta Lake. Every Databricks and Synapse pipeline you build will rely on it. Master that first.*

Quick Priority Reference

Priority Level	Topics
Non-Negotiable	Core Python, OOP, Error Handling, Logging, File I/O
Critical	Pandas, PySpark, Azure SDKs, DefaultAzureCredential, Key Vault
High	Delta Lake API, Databricks dbutils, pytest, Great Expectations, Airflow
Important	asyncio, pyodbc/sqlalchemy, REST API patterns, Docker, Poetry, Git
Good to Know	Polars, py-spy, Data Vault Python patterns, Unity Catalog SDK